```
#EL OTMANI Youssef worked with Omar
#Part 1

#1
v1 = vector( [3,5,4,2,1])
v2 = vector( [1,3,3,4,5])
print("v1", v1,"v2 =", v2, "\n")

#2
p= 5*v1-3*v2
print("5v1-3v2 =", p%2 , "\n")

#3
print("norme of v1 = ", v1.norm(), "\n")
print("norme of v2 = ", v2.norm(), "\n")

#4
print("dot product v1 and v2 = ", v1.dot_product(v2), "\n" )

#5
def proj(v,u):
    return (v.dot_product(u)/(norm(u)^2))*u

print("projection of v1 and v2 = ",proj(v1,v2), "\n")

#Part 2

#1
u,v,w,x,y,z=var('u,v,w,x,y,z')

#2a
H= [
    u +2*v -2*w +2*x -y +3*z == 0,
    2*u -v +3*w +x -3*y +2*z == 17,
    u +3*v -2*w +x -2*y -3*z == -5,
    3*u -2*v +w -x +3*y -2*z == -1,
    -u -2*v +w +2*x -2*y +3*z == 10,
    u -3*v +w +3*x -2*y +z == 11
]
print("equations = ", H, "\n")
print("solved = ", solve( H, [u,v,w,x,y,z]), "\n" )

#2b
A = matrix( [
    [1, 2, -2, +2, -1, 3],
    [2, -1, 3, 1, -3, 2],
    [1, 3, -2, 1, -2, -3],
    [3, -2, 1, -1, 3, -2],
    [-1, -2, 1, 2, -2, 3],
    [1, -3, 1, 3, -2, 1]
]
)
b = vector([0,17, -5, -1, 10, 11])
print("matrix is \n", A, "\n", "vector is", b , "\n")
print("solved = ", A.solve_right(b), "\n")

#2c
def column_replace (M , col , v ) :
    n1 = M . nrows ()
    n2 = M . ncols ()
    S = matrix( n1 , n2 )
    for i in range ( n1 ) :
        for j in range ( n2 ) :
            S[i , j ]= M [i , j ]
            #copy the matrix in a new matrix S
    for i in range ( n1 ) :
        S[i , col-1]= v[i ,0]
    return S

b = matrix (6 ,1 ,[0,17, -5, -1, 10, 11])
A1 = column_replace (A ,1 , b )
A2 = column_replace (A ,2 , b )
A3 = column_replace (A ,3 , b )
A4 = column_replace (A ,4 , b )
A5 = column_replace (A ,5 , b )
A6 = column_replace (A ,6 , b )

sol = matrix (6 ,1 ,[ det ( A1 ) / det ( A ) , det ( A2 ) / det ( A ) , det ( A3 ) / det
( A ) , det ( A4 ) / det ( A ), det(A5)/det(A), det(A6) / det(A) ])

print("solution using Cramer's rule", sol, "\n")

#Part 3

#1
D=QQ^5
v1 = vector([1,2, 3, 4, 5])
v2 = vector([-2,-4,-9,0,0])
v3 = vector([-3,-6,-15,4,5])
v4 = vector([0,0,1,0,0])
print("the vectors are = \n", v1 , "\n", v2, "\n", v3, "\n",v4, "\n")

#1a
value = D.linear_dependence( [v1,v2,v3,v4] ) == []
print("Are they linearly independent ? ",value, "\n")

#1b
W = D.span([v1,v2,v3,v4])
basis = W.basis()
print("Basis for span of these vectors = ", basis, "\n")

#1c
dim = W.dimension()
print("Dimension for span of these vectors = ", dim, "\n")

#1d
```

```
A = column_matrix ([ v1 , v2 ,v3, v4])
print("Matrix is \n", A, "\n")

#1e
A2=A.rref()
print("Reduced row echelon form of A is \n", A2, "\n")

#Part 4

#1
def Gram_Schmidt ( S ) :
    n = len( S )
    E =[ S[0]/ norm( S[0]).n()]
    for k in range (1 , n ) :
        q = S[k] - sum( [ proj(S[k],E[i]) for i in range( k ) ])
        E.append(q/norm( q ).n() )
    return (E)

b1 = vector(QQ, [7,24,0,0])
b2 = vector(QQ, [0,0,1,1])
b3 = vector(QQ, [0,0,1,-2])

B = [ b1, b2, b3]
print("The Base B ", B, "\n")
G = Gram_Schmidt(B)
print("Orthonormal basis from B ",G, "\n")

#2
a1 = vector(QQ, [1,1,1,1])
a2 = vector(QQ, [1,1,-1,-1])
a3 = vector(QQ, [0,-1,2,1])

A = [a1, a2, a3]
print("Another base ", A, "\n")
G2 = Gram_Schmidt(A)
print("Orthonormal basis ",G2, "\n")

#Part 5

#1
Dep = Arr = QQ^3
var('x1 , x2 , x3')
f( x1 , x2 , x3 ) =[ x1 + x2 + x3 , 2*x2 + 3*x3 , 4*x3 ]
T = linear_transformation(Dep , Arr , f )
print("the linear transformation T is ",T, "\n")

#2
u=vector(QQ, [2,2,2])
print("u is ", u, "and T(u) is ",T(u), "\n")

#3
print("kernel of T ",T.kernel(), "\n" )

#4
print("image of T ",T.image(), "\n" )

#5
print("injective ?", T.is_injective(), "\n")
print("surjective ?", T.is_surjective(), "\n")
print("bijective ?", T.is_bijective(), "\n")

#6
#in q3 we saw that the dimension of the kernal is 0 (kernal reduced to (0,0,0) )so that forces the function to be injective
#in Q4 we saw that the dimension of the image is 3, so that forces it to be all of R^3 so the function is surjective

#7
t1 = vector([1,2,3])
t2 = vector([3,3,3])
t3 = vector([0,0,5])

Base = [t1, t2, t3]
print("B is ",Base, "\n")

if(Dep.linear_dependence(Base) == []):
    print(Base," is lineraly independent \n")

#8
Dep1 = Dep.subspace_with_basis( Base )
S = T.restrict_domain( Dep1 ).restrict_codomain( Dep1 )
print(" matrix of T with respect to the basis B = \n",S.matrix( side = 'right') )

#Part 6

#1
A = matrix([[1 , 1 , 1] ,[0 , 2 , 3] ,[ 0 , 0 , 4]])

#1a
p(x) = A.characteristic_polynomial( x )
print("The characteristic polynomial is ", p(x), "\n")

#1b
print("The roots are ",p.roots(), "\n")

#1c
ev = A.eigenvalues()
print("The eigenvalues are ", ev, "\n")

#1d
vec = A.eigenvectors_right()
print("The eigenvector \n", vec, "\n")

#1e
h1= vec[0][1][0] #extracting just the vector from vec wich is a list with the eigenvalues and other informations
h2= vec[1][1][0]
h3= vec[2][1][0]
```

```
C = [h1, h2, h3]

print("C is ",C, "\n")
if( Dep.linear_dependence(C) == [] ) :
    print(C," is lineraly independent \n")

Dep2 = Dep.subspace_with_basis( C )

S2 = T.restrict_domain( Dep2 ).restrict_codomain( Dep2 )

print(" matrix of T with respect to the basis C = \n ",S2.matrix( side = 'right'), "\n")

#2
M = matrix([[3, -1 , -1 , 1] ,[0, 2 , 2 , 3] ,[ 0, 0 , 2 , 4], [ 0, 0 , 0 , 7] ])
print("matrix is \n", M, "\n")

#2a
v(x) = M.characteristic_polynomial( x )
print("polynom is ", v(x), "\n")

#2b
ev2 = M.eigenvalues()
print("eigenvalues = ", ev2, "\n")

#2c
vec2 = M.eigenvectors_right()
print("eigenvecotrs = ", vec2, "\n")

#2d
if len(ev2) == 4 :
    print("The matrix is diagonalisable")
```

```
Out :
v1 (3, 5, 4, 2, 1) v2 = (1, 3, 3, 4, 5)

5v1-3v2 = (0, 0, 1, 0, 0)

norme of v1 =  sqrt(55)

norme of v2 =  2*sqrt(15)

dot product v1 and v2 =  43

projection of v1 and v2 =  (43/60, 43/20, 43/20, 43/15, 43/12)

equations =  [u + 2*v - 2*w + 2*x - y + 3*z == 0, 2*u - v + 3*w + x - 3*y + 2*z == 17, u + 3*v - 2*w + x - 2*y - 3*z == -5, 3*u - 2*v + w - x + 3*y - 2*z == -1

solved =  [
[u == 1, v == -1, w == 2, x == 0, y == -2, z == 1]
]

matrix is
 [ 1  2 -2  2 -1  3]
[ 2 -1  3  1 -3  2]
[ 1  3 -2  1 -2 -3]
[ 3 -2  1 -1  3 -2]
[-1 -2  1  2 -2  3]
[ 1 -3  1  3 -2  1]
 vector is (0, 17, -5, -1, 10, 11)

solved =  (1, -1, 2, 0, -2, 1)

solution using Cramer's rule [ 1]
[-1]
[ 2]
[ 0]
[-2]
[ 1]

the vectors are =
 (1, 2, 3, 4, 5)
 (-2, -4, -9, 0, 0)
 (-3, -6, -15, 4, 5)
 (0, 0, 1, 0, 0)

Are they linearly independent ?  False

Basis for span of these vectors =  [
(1, 2, 0, 0, 0),
(0, 0, 1, 0, 0),
(0, 0, 0, 1, 5/4)
]

Dimension for span of these vectors =  3

Matrix is
 [  1  -2  -3   0]
[  2  -4  -6   0]
[  3  -9 -15   1]
[  4   0   4   0]
[  5   0   5   0]

Reduced row echelon form of A is
 [1 0 1 0]
[0 1 2 0]
[0 0 0 1]
[0 0 0 0]
[0 0 0 0]

The Base B  [(7, 24, 0, 0), (0, 0, 1, 1), (0, 0, 1, -2)]

Orthonormal basis from B  [(0.280000000000000, 0.960000000000000, 0.000000000000000, 0.000000000000000), (0.000000000000000, 0.000000000000000, 0.7071067811865

Another base  [(1, 1, 1, 1), (1, 1, -1, -1), (0, -1, 2, 1)]

Orthonormal basis  [(0.500000000000000, 0.500000000000000, 0.500000000000000, 0.500000000000000), (0.500000000000000, 0.500000000000000, -0.500000000000000, -0
```

```
the linear transformation T is  Vector space morphism represented by the matrix:
[1 0 0]
[1 2 0]
[1 3 4]
Domain: Vector space of dimension 3 over Rational Field
Codomain: Vector space of dimension 3 over Rational Field

u is  (2, 2, 2) and T(u) is  (6, 10, 8)

kernel of T  Vector space of degree 3 and dimension 0 over Rational Field
Basis matrix:
[]

image of T  Vector space of degree 3 and dimension 3 over Rational Field
Basis matrix:
[1 0 0]
[0 1 0]
[0 0 1]

injective ? True

surjective ? True

bijective ? True

B is  [(1, 2, 3), (3, 3, 3), (0, 0, 5)]

[(1, 2, 3), (3, 3, 3), (0, 0, 5)]  is lineraly independent

 matrix of T with respect to the basis B =
 [   7    6   10]
[-1/3    1 -5/3]
[-8/5 -9/5   -1]
The characteristic polynomial is  x^3 - 7*x^2 + 14*x - 8

The roots are  [(2, 1), (4, 1), (1, 1)]

The eigenvalues are  [4, 2, 1]

The eigenvector
 [(4, [
(1, 9/5, 6/5)
], 1), (2, [
(1, 1, 0)
], 1), (1, [
(1, 0, 0)
], 1)]

C is  [(1, 9/5, 6/5), (1, 1, 0), (1, 0, 0)]

[(1, 9/5, 6/5), (1, 1, 0), (1, 0, 0)]  is lineraly independent

 matrix of T with respect to the basis C =
  [4 0 0]
[0 2 0]
[0 0 1]

matrix is
 [ 3 -1 -1  1]
[ 0  2  2  3]
[ 0  0  2  4]
[ 0  0  0  7]

polynom is  x^4 - 14*x^3 + 65*x^2 - 124*x + 84

eigenvalues =  [7, 3, 2, 2]

eigenvecotrs =  [(7, [
(1, -46/9, -40/9, -50/9)
], 1), (3, [
(1, 0, 0, 0)
], 1), (2, [
(1, 1, 0, 0)
], 2)]

The matrix is diagonalisable
```